

ProjectFlow API Documentation

Comprehensive REST API Reference Guide

Authentication & Authorization

All endpoints (except /auth/register, /auth/login, and /auth/explore) require a valid JWT token passed in the Authorization header:

```
Authorization: Bearer <token>
```

1. Register Account

POST /auth/register

- **Body:** { "name": "Wesal", "username": "user1", "password": "password123" }
 - name, username, and password are required fields.
 - password must be at least 6 characters long.
- **Response (201):** { "message": "...", "recoveryKey": "..." }

2. Login

POST /auth/login

- **Body:** { "username": "user1", "password": "password123" }
- **Response (200):** { "token": "...", "message": "..." }

3. Explore Mode

POST /auth/explore

- Returns a token for the isolated demo user.
- **Response (200):** { "token": "...", "message": "..." }

4. Get Current User

GET /auth/me

- **Response (200):** { "id": 1, "name": "Wesal", "username": "user1", "isDemo": false }

5. Update Profile

PATCH /auth/me

- **Body:** { "name": "New Name" }
- **Response (200):** { "message": "Profile updated successfully", "name": "New Name" }

6. Delete Account

DELETE /auth/me

- **Response (200):** { "message": "Account deleted successfully" }

7. Reset Password

POST /auth/reset-password

- **Body:** { "username": "user1", "recoveryKey": "the-recovery-key", "newPassword": "newpassword" }
- Validates that newPassword is at least 6 characters.
- **Response (200):** { "message": "Password reset successfully" }
- **Error (401):** Invalid username or recovery key.
- *Note:* This endpoint does NOT require a JWT token.

8. Logout

POST /auth/logout

- **Response (200):** { "message": "..." }

User Data Structure

The User model in the database has the following schema:

- id (INTEGER PRIMARY KEY)
- name (TEXT)
- username (TEXT)
- password (TEXT, hashed via bcrypt, not returned to client)
- recoveryKey (TEXT, only exposed upon registration)
- isDemo (BOOLEAN)

Projects

Get All Projects

GET /projects

- **Query Parameters:**
 - filter: all, active, overdue, cancelled, completed (default: all)
 - sortOrder: asc, desc (default: asc)
- **Response (200):** Array of Project objects with derivedStatus.

Get Project by ID

GET /projects/:id

- **Response (200):** Project object

Create Project

POST /projects

- **Body:** { "name": "...", "description": "...", "dueDate": "YYYY-MM-DD", "cancelled": false, "techStack": ["React"] }
- **Response (201):** { "id": 1, "message": "..." }

Update Project

PUT /projects/:id

- **Body:** Same as Create (all fields optional)
- **Response (200):** { "message": "..." }

Delete Project

DELETE /projects/:id

- **Response (200):** { "message": "..." }

Tasks

Get Tasks

GET /tasks

- **Query Parameters:**

- projectId (optional)
- status: all, open, completed, overdue (default: all)
- priority: all, high, medium, low (default: all)
- sortOrder: asc, desc (default: asc)
- page: Number (default: 1)
- limit: Number (default: 10)

- **Response (200):** { "data": [...], "pagination": { "currentPage": 1, "limit": 10, "totalItems": 5, "totalPages": 1 } }

Get Task by ID

GET /tasks/:id

- **Response (200):** Task object

Create Task

POST /tasks

- **Body:** { "name": "...", "projectId": 1, "status": "open", "priority": "high", "dueDate": "YYYY-MM-DD", "description": "..." }
- **Response (201):** { "id": 1, "message": "..." }

Update Task

PUT /tasks/:id

- **Body:** Same as Create (all fields optional)
- **Response (200):** { "message": "..." }

Delete Task

DELETE /tasks/:id

- **Response (200):** { "message": "..." }

Notes

Get Notes for Task

GET /tasks/:taskId/notes

- **Response (200):** Array of Note objects

Get Note by ID

GET /notes/:id

- **Response (200):** Note object

Create Note

POST /notes

- **Body:** { "content": "...", "taskId": 1 }
- **Response (201):** { "id": 1, "message": "..." }

Update Note

PUT /notes/:id

- **Body:** { "content": "..." }
- **Response (200):** { "message": "..." }

Delete Note

DELETE /notes/:id

- **Response (200):** { "message": "..." }

Attachments

Get Attachments for Project

GET /projects/:projectId/attachments

- **Response (200):** Array of Attachment metadata objects

Get Attachment Metadata

GET /attachments/:id

- **Response (200):** Attachment metadata object

Upload Attachment

POST /attachments

- **Format:** multipart/form-data
- **Fields:**
 - file: The actual file (allowed types: png, jpg, jpeg, svg, pdf, md, txt)
 - projectId: The ID of the project
- **Response (201):** { "id": 1, "message": "..."}

Download Attachment

GET /attachments/:id/download

- **Response (200):** File stream

Delete Attachment

DELETE /attachments/:id

- **Response (200):** { "message": "..."}